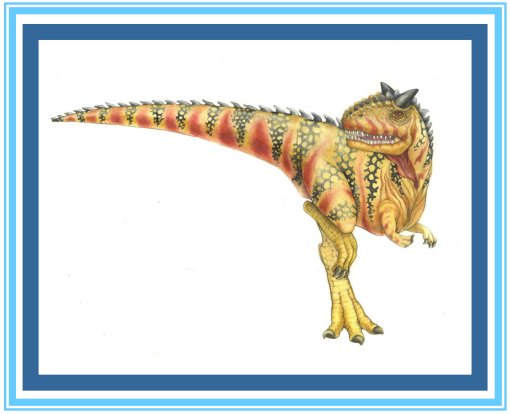
Chapter 6: Synchronization



Silberschatz, Galvin and Gagne ©2013



Chapter 6: Synchronization Tools

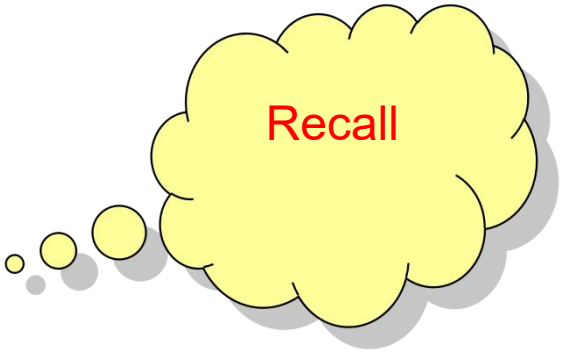
- 6.1 Background
- 6.2 The Critical-Section Problem
- 6.3 Peterson's Solution
- 6.4 Synchronization Hardware
- 6.5 Mutex Locks
- 6.6 Semaphores

OBJECTIVES

- To present the concept of process synchronization.
- To introduce the critical-section problem, whose solutions can be used to ensure the consistency of shared data
- To present both software and hardware solutions of the critical-section problem
- To examine several classical process-synchronization problems
- To explore several tools that are used to solve process synchronization problems



Introduction



- *Independent* process
  - A process that does not share data with other processes.
  - cannot affect or be affected by the execution of another process.

- *Cooperating* process
  - can affect or be affected by the execution of another process.
  - processes directly share logical address space (code & data) or processes share data through files or messages.

Silberschatz, A., Galvin, P.B., and Gagne, G. Operating System Concepts (9th Edition). John Wiley & Sons: Asia. (2014) Page 120.



6.1 Background

- **Concurrent access:** accessing of the same data at the same time by more than one process, thus:
  - Concurrent access to shared data may result in data inconsistency
  - Maintaining data consistency requires mechanisms to ensure the orderly execution of cooperating processes
  - **Race Condition**: problem in concurrent execution
- The synchronization mechanism is usually provided by both hardware and the operating system
- Illustration of the problem – The producer-Consumer problem.
- Basic assumption – load and store instructions are atomic.



Why a Race Condition Occurs?

Two processes may be :

- executing simultaneously, and
- trying to access the same global variable.





A Simple Example

The fundamental problem in concurrency is processes interfering with each other while accessing a shared global resource. This can be illustrated with a surprisingly simple example:

```
chin = getchar();
chout = chin;
putchar(chout);
```



http://undergraduate.csse.uwa.edu.au/units/CITS2230/handouts/Lecture09/lecture9.pdf



```
chin = getchar();
chout = chin;
putchar(chout);
```

Imagine two processes **P1** and **P2** both executing this code at the "same" time, with the following interleaving due to multi-programming.

1. **P1** enters this code, but is interrupted after reading the character x into **chin**.
2. **P2** enters this code, and runs it to completion, reading and displaying the character y.
3. **P1** is resumed, *but* **chin** now contains the character y, so **P1** displays the wrong character.

The essence of the problem is the shared global variable **chin**. **P1** sets **chin**, but this write is subsequently lost during the execution of **P2**. The general solution is to allow only one process at a time to enter the code that accesses **chin**: such code is often called a Critical section When one process is inside a critical section of code, other processes must be prevented from entering that section. This requirement is known as Mutual exclusion



Producer-Consumer Problems

- Several processes work together to complete common task – ***Process Cooperation***
- Common paradigm for cooperating processes, ***producer*** process produces information that is consumed by a ***consumer*** process.

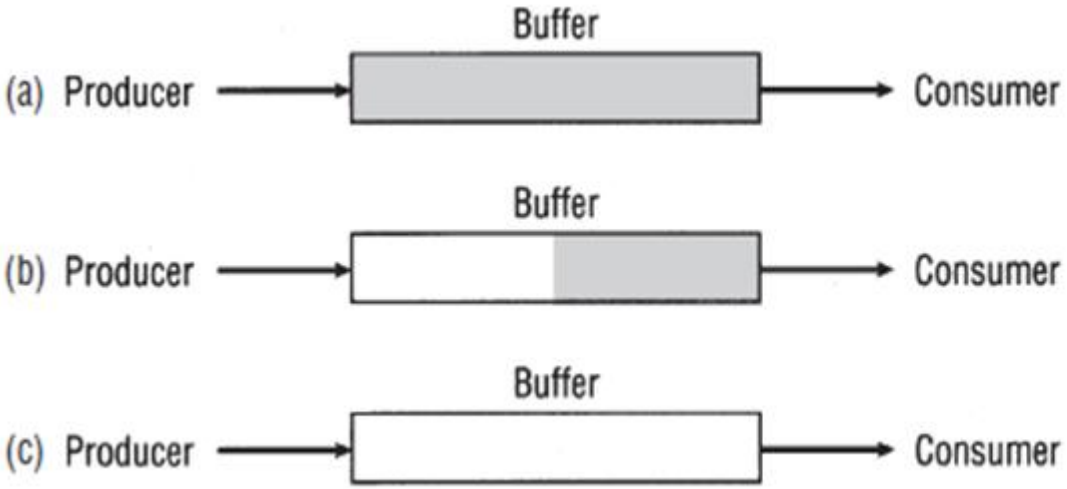
Examples:

- The compiler **produces** an assembly code which is **consumed** by an assembler.
- The web server **provides** HTML files which is **consumed** by the client web browser requesting the resource.

- Suppose that we wanted to provide a solution that fills ***all*** the buffers where we allow the producer and consumer processes to increment and decrement the same variable.
- We can do so by adding another integer variable – a **`counter`** that keeps track of the number of full buffers. Initially, **`counter = 0`**.
- The variable **counter,** is incremented by the producer after it produces a new buffer and is decremented by the consumer after it consumes a buffer.
- The code is shown in the next two slides

- To enable ***producer*** and ***consumer*** to be executed concurrently, a ***buffer*** (shared memory) is used for producer to put data while consumer access it.

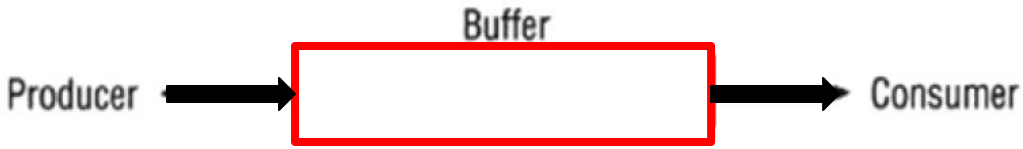
  - ***unbounded-buffer*** provides unlimited buffer size.
  - ***bounded-buffer*** assumes that there is a fixed buffer size.



(figure 6.5)

The buffer can be in any one of these three states: (a) full buffer, (b) partially empty buffer, or (c) empty buffer.

- ***Producer*** puts data in one slot while ***consumer*** retrieve data from another slot.
- ***Producer*** and ***consumer*** need to be synchronized so that ***consumer*** will not retrieve data before ***producer*** puts the data into the buffer.



- Therefore,

  - **producer** needs to **wait** if buffer is **full**
  - **consumer** needs to **wait** if buffer is **empty**



Producer Process

```
while (true) {
        /* produce an item in next produced */

        while (counter == BUFFER_SIZE) ;
                /* do nothing */
        buffer[in] = next_produced;
        in = (in + 1) % BUFFER_SIZE;
        counter = counter +1;
}
```





Consumer Process

```
while (true) {
        while (counter == 0)
                ; /* do nothing */
        next_consumed = buffer[out];
        out = (out + 1) % BUFFER_SIZE;
       counter = counter - 1;

        /* consume the item in next consumed */

}
```



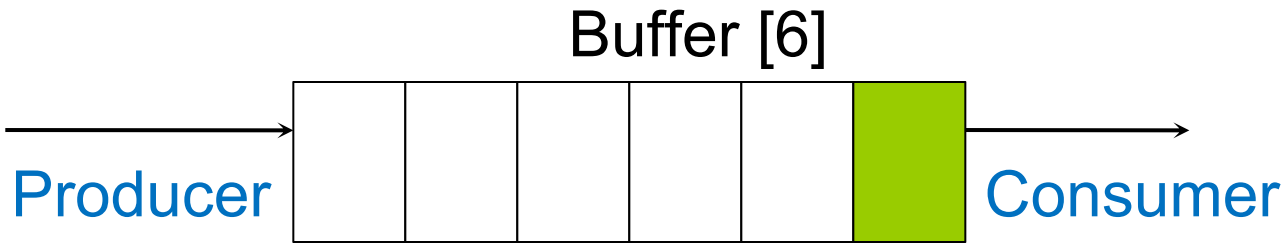
```
#define buffer_size 10
      typedef struct { … } item;
      item buffer[buffer_size];
      int  in=0;
      int  out=0;
      int  counter=0;
```

Bounded-Buffer Problems

- Shared Variables
  - `in`, `out`, `counter` and `buffer[ ]`
  - Initial values for `in`, `out`, `counter` =0
- Both routines are correct if executed independently.
- However, it may give problem if run concurrently when the processes are not synchronized properly.

(For Slide page 21-24:
Silberschatz, A., Galvin, P.B., and Gagne, G. Operating System Concepts (8th Edition). John Wiley & Sons: Asia. (2010) Page 254.





- If current counter = 5 and statements:

  `counter ++` → (producer)

  `counter --` → (cons umer)

  are executed concurrently . . .

  - Then the counter maybe 4, 5 or 6.
  - The correct value is 5 (synchronized properly)





- Each statement consists of several machine instructions:

```
counter ← counter + 1
    reg1 ← counter
    reg1 ← reg1 + 1
    counter ← reg1
```

Producer
counter++

```
counter ← counter - 1
    reg2 ← counter
    reg2 ← reg2 - 1
    counter ← reg2
```

Consumer
counter--





Race Condition

- **`counter = counter + 1`** could be implemented as

```
register1 = counter
register1 = register1 + 1
counter = register1
```

- **`counter = counter -1`** could be implemented as

```
register2 = counter
register2 = register2 - 1
counter = register2
```

- Consider this execution interleaving with "count = 5" initially:

S0: producer execute `register1 = counter` {register1 = 5}
S1: producer execute `register1 = register1 + 1` {register1 = 6}
S2: consumer execute `register2 = counter` {register2 = 5}
S3: consumer execute `register2 = register2 - 1` {register2 = 4}
S4: producer execute `counter = register1` {counter = 6 }
S5: consumer execute `counter = register2` {counter = 4}

Synchronize problem (race condition)



Race Condition - Definition

- Def 1: A situation whereby at least **two processes** perform some operations on shared data and the outcome depends on the order of data access.
- Result of process must be **independent of the speed of execution** of other concurrent processes;
  - Depend on the control / data synchronization mechanisms.
- Def 2: A situation whereby **multiple threads** access a data item without coordination in a multithreaded application.
- possibly causing inconsistent results (depending on which thread reaches the data item first).

** (Multithread normally access / used shared variables)*

Silberschatz, A., Galvin, P.B., and Gagne, G. Operating System Concepts (9th Edition). John Wiley & Sons: Asia. (2014) Page 255.

Solution to Race Condition

- Ensure that only one process at a time can manipulate the shared variable (eg. `Counter`)
- Therefore ***process synchronization*** is needed.
  - means coordinating the activities of two or more processes.
- Synchronization is necessary to ensure that interdependent code is executed in the **proper sequence**.



Race Condition (Cont.)

- How do we solve the race condition?

- We need to make sure that:
  - The execution of

    ```
    counter = counter + 1
    ```

    is executed as an “atomic” action. That is, while it is being executed, no other instruction can be executed concurrently.
  - Similarly for

    ```
    counter = counter - 1
    ```

- The ability to execute an instruction, or a number of instructions, atomically is crucial for being able to solve many of the synchronization problems.



Extra

Exercise 6.1:

Based on a several machine instructions below that has synchronization problem, give your suggestion on how to overcome that race condition situation.

T_0	Producer	`reg1` ← counter	{`reg1 = 5`}
T_1	Producer	`reg1` ← `reg1+1`	{`reg1 = 6`}
T_2	Consumer	`reg2` ← counter	{`reg2 = 5`}
T_3	Consumer	`reg2` ← `reg2-1`	{`reg2 = 4`}
T_4	Producer	counter ← `reg1`	{`counter=6`}
T_5	Consumer	counter ← `reg2`	{`counter=4`}

Extra

Solution 6.1:

Producer should update the counter just before consumer read the value of counter.

T_0	Producer	reg1 ← counter	{reg1 = 5}
T_1	Producer	reg1 ← reg1+1	{reg1 = 6}
T_2	Producer	counter ← reg1	{counter=6}
T_3	Consumer	reg2 ← counter	{reg2 = 6}
T_4	Consumer	reg2 ← reg2-1	{reg2 = 5}
T_5	Consumer	counter ← reg2	{counter=5}



6.2 Critical Section Problem

- Part of a program (segment of code for each process)
- Consider system of ***n*** processes {$P_0, P_1, \ldots P_{n-1}$}
- Each process has **critical section** segment of code
  - Process may be changing common variables, updating table, writing file, etc
  - When one process in critical section, no other may be in its critical section
- ***Critical section problem*** is to design protocol to solve this.
- Each process must ask permission to enter critical section in **entry section** code; it than executes in the critical section; once it finishes executing in the critical section it enters the **exit section** code. The process then enters the **remainder section** code.





General structure of Process Entering the Critical Section

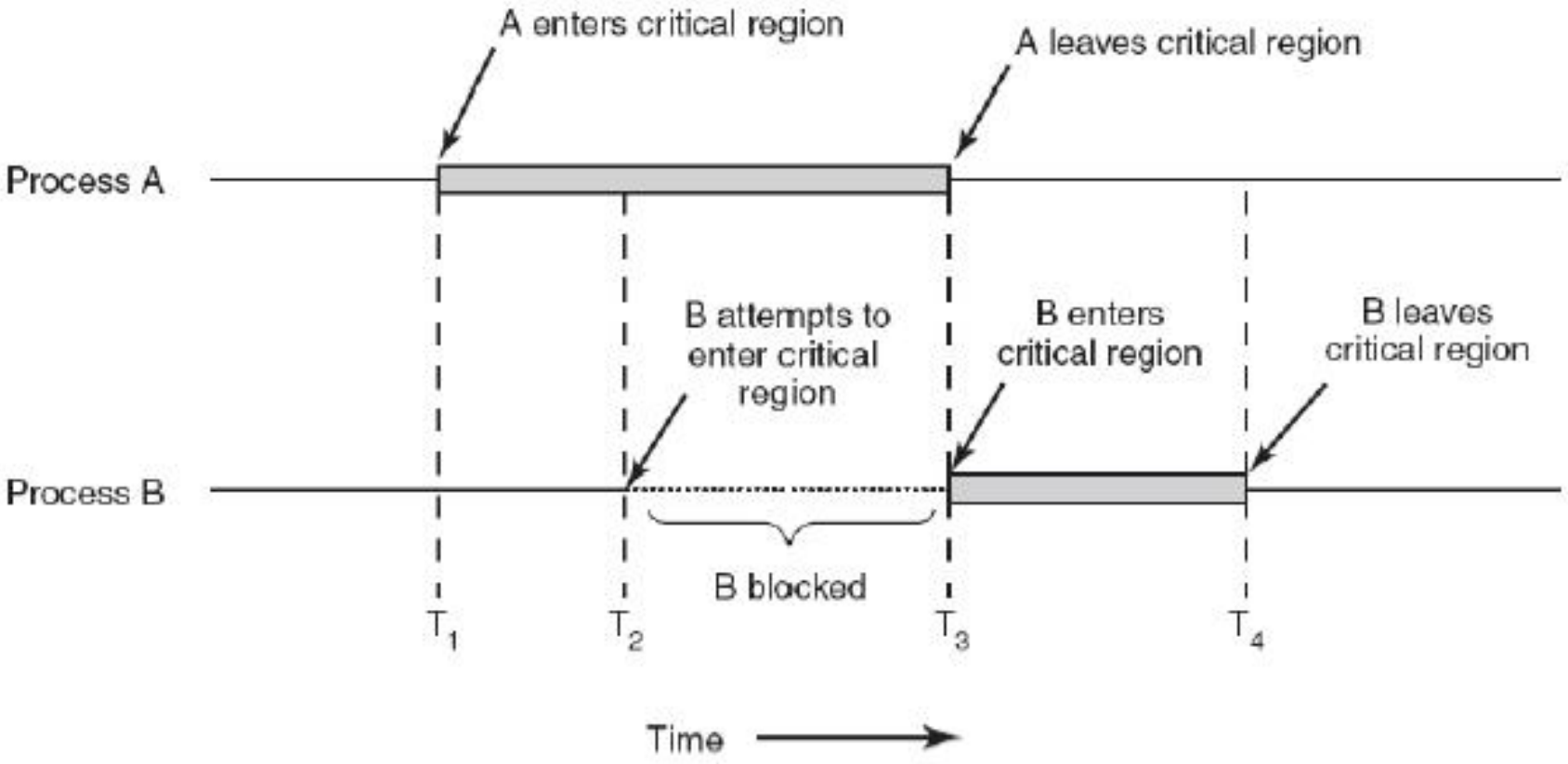
- General structure of process P_i

```
do {
    entry section
        critical section
    exit section
        remainder section
} while (true);
```

Figure 6.1 General structure of Process Entering the Critical Section



CS – Process Structure





Algorithm for Process P_i

- Entry section to critical section is to “disable interrupts.
- Execute in the critical section
- Exit section is to “enable interrupts”.
- Implementation issues:
  - Uniprocessor systems
    - Currently running code would execute without preemption
  - Multiprocessor systems.
    - Generally too inefficient on multiprocessor systems
    - Operating systems using this not broadly scalable
- Is this an acceptable solution?
  - This is impractical if the critical section code is taking a long time to execute.





Algorithm for Process P_i

Have a variable "turn" to indicate which process is next

```
do {

    while (turn == j);

        critical section

    turn = j;

        remainder section

} while (true);
```

Figure 6.2

- Algorithm is correct. Only one process at a time in the critical section.
- The other process will do *Busy waiting for until* the current process finish its critical section.





Requirements for CS solutions

Assumptions:

- Assume that each process executes at a non-zero speed
- No assumption concerning relative speed of the *n* processes or #*number* of CPU.

The algorithm does satisfy the three essential criteria to solve the CS problems.

Mutual Exclusion

Progress

Bounded Waiting





Solution to Critical-Section Problem

Must satisfy the following three requirements:

1. **Mutual Exclusion** - If process P_i is executing in its critical section, then no other processes can be executing in their critical sections
2. **Progress** - If no process is executing in its critical section and there exist some processes that wish to enter their critical section, then the selection of the processes that will enter the critical section next cannot be postponed indefinitely
3. **Bounded Waiting** - A bound must exist on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted
   - Assume that each process executes at a nonzero speed
   - No assumption concerning **relative speed** of the ***n*** processes



4 Types of locking mechanisms to solve CS problem:

1. Software defined (*Strict Alternation, Flagging and Peterson*)

2. Hardware support (*Test-and-set and swap*)

3. Support from the Operating System (*Semaphore*)

4. Support the programming language (Ada – rendevous)



6.3 Software Solution: Peterson's Algorithm

- Good algorithmic software solution
- Two process solution
- Assume that the **`load`** and **`store`** machine-language instructions are atomic; that is, cannot be interrupted
- The two processes share two variables:
  - **`int turn;`**
  - **`Boolean flag[2]`**
- The variable **`turn`** indicates whose turn it is to enter the critical section
- The **`flag`** array is used to indicate if a process is ready to enter the critical section. **`flag[i] = true`** implies that process P_i is ready!





Algorithm for Process P_i

```
do {
    flag[i] = true;
    turn = j;
    while (flag[j] && turn = = j);
            critical section
    flag[i] = false;
            remainder section
} while (true);
```

Figure 6.2 The structure of Process P_i in Peterson's Solution

- Provable that the three CS requirements are met:

1. Mutual exclusion is preserved

   P_i enters CS only if:

   either **flag[j] = false** or **turn = i**

2. Progress requirement is satisfied

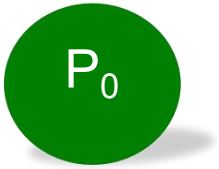
3. Bounded-waiting requirement is met



Exercises 1

- Rewrite the Peterson's algorithm for CS solution for *process 0* (P_0).
- Rewrite the Peterson's algorithm for CS solution for *process 1* (P_1).







Initial SHARED values:
flag[0] = FALSE
flag[1] = FALSE

Answer

```
do {
     flag[0] = TRUE;
     turn = 1;
     while(flag[1] && turn == 1)
            ;  /* do nothing  */
         critical section
     flag[0] = FALSE;
         remainder section
} while (TRUE);
```

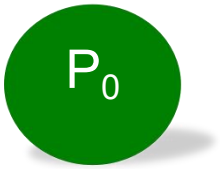
P_0

```
do {
     flag[1] = TRUE;
     turn = 0;
     while(flag[0] && turn == 0)
            ;  /* do nothing  */
         critical section
     flag[1] = FALSE;
         remainder section
} while (TRUE);
```

P_1

Initially the flags are false. When a process wants to execute it's critical section, it sets it's flag to true and turn as the index of the other process. This means that the process wants to execute but it will allow the other process to run first. The process performs busy waiting until the other process has finished it's own critical section. After this the current process enters it's critical section and adds or removes a random number from the shared buffer. After completing the critical section, it sets it's own flag to false, indication it does not wish to execute anymore.

"I am ready to enter CS"
"But you can execute your own CS"
"If you are ready to enter and it's your turn I'll wait."
Otherwise, I'll enter my CS
"I don't want to enter any more."




```
flag[0]=true;
turn=1;
while(flag[1]==true&&turn==1){
/*P0 wait*/        }
     CS
flag[0]=false;
```

Entrance to the CS is granted for process P_0

- if P_1 does not want to enter its CS,

```
flag[1]=FALSE
```

or

- if P_1 has given priority to P_0 by setting `turn=0`.

Exercises 2

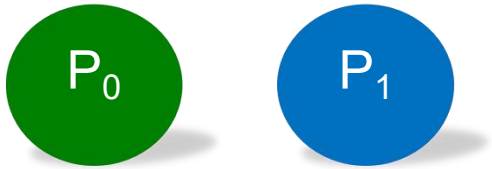
- What is the event for each t_i if 2 concurrent processes wishing to enter their CSs.



time	flag[0]	flag[1]	turn
t_0	TRUE	FALSE	1
t_1	TRUE	FALSE	1
t_2	TRUE	TRUE	0
t_3	FALSE	TRUE	0
t_4	FALSE	TRUE	0
t_5	FALSE	TRUE	0
t_6	FALSE	FALSE	0
t_7	FALSE	FALSE	0
t_8	FALSE	TRUE	0

- Check if the following is preserved:
  - Mutual exclusion
  - Progress
  - Bounded Waiting

Answer: Execution of algorithm



time	flag[0]	flag[1]	turn	Events
t_0				
t_1				
t_2				
t_3				
t_4				
t_5				
t_6				
t_7				
t_8				

Answer: Execution of algorithm



Mutual Exclusion

	flag[0]	flag[1]	turn	Events
	TRUE	FALSE	1	P_0 request to enter CS_0
t_1	TRUE	FALSE	1	P_0 enters CS_0
t_2	TRUE	**TRUE**	0	P_1 requests to enter CS_1
t_3	**FALSE**	TRUE	0	P_0 executes RS_0
t_4	FALSE	TRUE	0	P_1 enters CS_1
t_5	FALSE	TRUE	0	P_0 executes RS_0
t_6	FALSE	**FALSE**	0	P_1 executes RS_1
t_7	FALSE	FALSE	0	P_0 executes RS_0
t_8	FALSE	**TRUE**	0	P_1 requests to enter CS_1

Peterson's algorithm :

Table shows a process that can enter its CS for different values of **`flag[]`** and **`turn`**

`flag[0]`	`flag[1]`	`turn`	Process that can enter its CS
TRUE	FALSE	1	P_0
FALSE	TRUE	0	P_1
TRUE	TRUE	0	P_0
TRUE	TRUE	1	P_1



Exercise 3: Continue from previous example

time	flag[0]	flag[1]	turn	Events
t_{10}	**TRUE**	FALSE	1	P_0 requests to enter CS_0
t_{11}	TRUE	**TRUE**		
t_{12}	TRUE	TRUE		
t_{13}	TRUE	TRUE		
t_{14}	**FALSE**	TRUE		
t_{15}	FALSE	TRUE		
t_{16}	**TRUE**	TRUE		
t_{17}	TRUE	TRUE		
t_{18}	TRUE	**FALSE**		
t_{19}	TRUE	FALSE		



Exercise 3: Continue from previous example

time	flag[0]	flag[1]	turn	Events
t_{10}	**TRUE**	FALSE	1	P_0 requests to enter CS_0
t_{11}	TRUE	**TRUE**	0	P_1 request to enter CS_1
t_{12}	TRUE	TRUE	0	P_0 enters CS_0
t_{13}	TRUE	TRUE	0	P_1 busy waiting in loop
	E	TRUE	0	P_0 execut
t_{15}	FALSE	TRUE	0	P_1 ent
t_{16}	**TRUE**	TRUE	1	P_0 requ
t_{17}	TRUE	TRUE	1	P_0 busy waiting in loop
t_{18}	TRUE	**FALSE**	1	P_1 executes RS_1
t_{19}	TRUE	FALSE	1	P_0 enters CS_0

Progress

Bounded Waiting



Solution to Critical-section Problem Using Locks

- Software-based solutions such as Peterson's are not guaranteed to work on modern computer architectures.
- Many systems provide hardware support for implementing the critical section code.
- All solutions are based on idea of **locking**
  - Protecting critical regions via locks
- Code:

```
do {
        acquire lock

                critical section

        release lock

                remainder section

} while (TRUE);
```





Synchronization Hardware

- Modern machines provide special atomic hardware instructions to implement locks
  - **Atomic** = non-interruptible
- Two types instructions:
  - Test memory word and set value
  - Swap contents of two memory words



Test-and-Set Instruction

- **Test-and-Set** is a single indivisible machine instruction known simply as **TS** and was introduced by IBM for its multiprocessing System 360/370 computers.

- In a single machine cycle, it tests to see if the key is available and if it is, sets it to unavailable.

- The key is a single bit in a storage location that can contain
  - `0` (if it is free) or
  - `1` (if busy).



test_and_set Instruction

- Definition:

```
boolean test_and_set (boolean *target)
    {
        boolean rv = *target;
        *target = TRUE;
        return rv:
    }
```

Figure 6.3 The definition of the *test_and_set()* instruction

- **Properties:**
  - Executed atomically
  - Returns the original value of passed parameter
  - Set the new value of passed parameter to “TRUE”.



Test-and-Set for CS Application

- a process (P_1) would test the condition code using **Test-and-Set** instruction before entering a CS.
- If no other process was in the CS, then P_1 would be allowed to proceed and the condition code is would be changed from `0` to `1`.
- Later, when P_1 exits the CS, the condition code is reset to `0` so that another process can enter its CS.
- On the other hand, if P_1 finds a busy condition code, then it is placed in a waiting loop where it continues to **test** the condition code and wait until it is free.



Solution using test_and_set()

- Shared Boolean variable `lock`, initialized to FALSE
- Each process, wishing to execute CS code:

```
do {
        while (test_and_set(&lock))
        ; /* do nothing */
            /* critical section */
    lock = false;
            /* remainder section */
  } while (true);
```

Figure 6.4 Mutual exclusion implementation with *test_and_set()*

- Solution results in busy waiting.
- What about bounded waiting?
- However, this algorithm does not satisfy bounded waiting. Read textbook for the solution.





6.5 Mutex Locks

- Previous solutions are complicated and generally inaccessible to application programmers
- OS designers build software tools to solve critical section problem.
- Simplest tools is the **mutex lock,** which has a Boolean variable "available" associated with it to indicate if the lock is available or not.
- However, there is a more robust tool that can behave similarly to a mutex lock but can also provide more sophisticated ways for processes to synchronize their activities.





Semaphores

Overview

A method of visual signaling, usually by means of *flags* or *lights*.

They are used in :

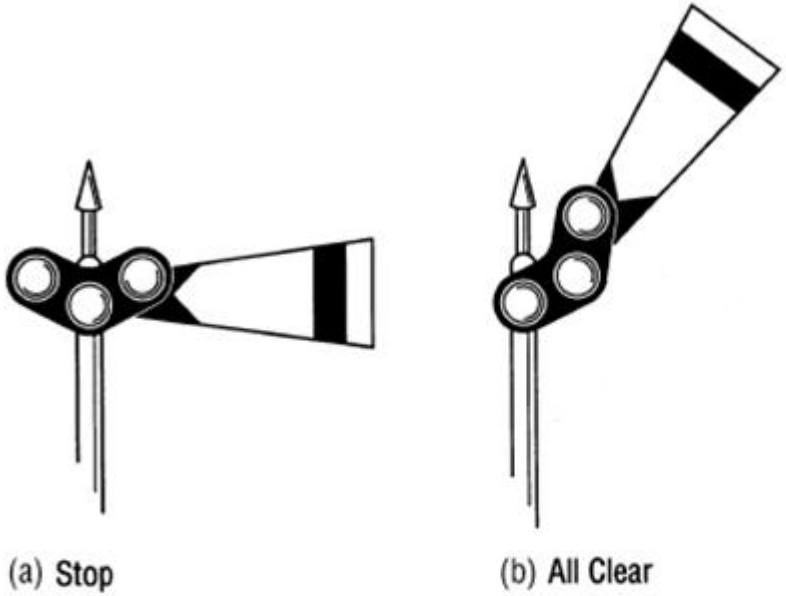
- railways (*raise arm* → *track is clear, lowered arm* → *track is busy* and *train must wait*)



- between foreign navy ships to communicate messages over long distances.



Semaphores





(figure 6.4)

The semaphore used by railroads indicates whether your train can proceed. When it's lowered (a), another train is approaching and your train must stop to wait for it to pass. If it is raised (b), your train can continue.

Similar function in OS i.e it signals if and when a resource is free and can be used by a process.





Semaphores

Semaphore in OS:

- Protocol mechanism for task communication.
- Specifically, semaphores are used to:
  - Control access to a shared resource (mutual exclusion).
  - Signal the occurrence of an event.
  - Allow two tasks to synchronize their activities.





Semaphores

- Synchronization tool that provides more sophisticated ways (than Mutex locks) for processes to synchronize their activities.
- Semaphore ***S*** – integer variable
- Can only be accessed via two indivisible (atomic) operations
  - when one process modify semaphore variable S, then **no other process can modify** the variable concurrently → **mutual exclusion**
  - **wait()** is called **P()**
  - and **signal()** called **V()**





Semaphores

- Definition of the **wait()** operation

```
wait(S)
{   while (S <= 0) ; // busy waiting
        S = S - 1;
 }
```

(S <= 0 implies busy critical section/region or a process in CS)

(Process calling on **wait()** operation must wait until **S > 0** (+ve value))

- Definition of the **signal()** operation

```
signal(S)
{
    S = S + 1;
 }
```





Types of Semaphores

- **Counting semaphore** – integer value can range over an unrestricted domain
- **Binary semaphore** – integer value can range only between 0 and 1
  - Same as a **mutex lock**
- Can implement a counting semaphore ***S*** as a binary semaphore





Semaphore Usage

Can solve various synchronization problems

- A solution to the CS problem.
  - Create a semaphore "**`synch`**" initialized to 1

```
wait(synch)
        CS
signal(synch);
```





Semaphore Usage

Consider P_1 and P_2 that require code segment S_1 to happen before code segment S_2

Create a semaphore "**synch**" initialized to 0

P1:

S_1;

signal(synch);

P2:

wait(synch);

S_2;

Because `synch` is initialized to 0, P_2 is executed only after P_1 has invoked `signal(sync)`, which is after statement S_1 has been executed.

S <= 0 implies busy critical section/region or a process in CS





Semaphore Implementation: Busy waiting

- Must guarantee that no two processes can execute the **`wait()`** and **`signal()`** on the same semaphore at the same time
- Thus, the implementation becomes the critical section problem where the **`wait`** and **`signal`** code are placed in the critical section
- This implementation is based on **busy waiting** in critical section implementation (that is, the code for **`wait()`** and **`signal()`**)
  - But implementation code is short
  - Little busy waiting if critical section rarely occupied
- Can we implement semaphores with no busy waiting? YES. The textbook has the answer.





Semaphore Implementation with no Busy Waiting

- With each semaphore there is an associated waiting queue
- Each entry in a waiting queue has two data items:
  - value (of type integer)
  - pointer to next record in the list

```
typedef struct{
int value;
struct process *list;
} semaphore;
```

- Two operations:
  - **block** – place the process invoking the operation on the appropriate waiting queue
  - **wakeup** – remove one of processes in the waiting queue and place it in the ready queue





Exercise:

The following codes consist of 2 semaphores and 2 concurrent processes. Assume that these processes run for a sufficiently long time under a preemptive scheduler.

```
struct semaphore S, V;
S = 2;
V = 0;
```

```
// Process 1
while (1) {
  wait(S);
  printf("Great \n");
  signal(V);
}
```

```
// Process 2
while (1) {
  wait(V);
  printf("day \n");
  printf ("today!! \n");
  signal(V);
}
```

Continue…

```
struct semaphore S, V;
S = 2;
V = 0;
```

// **Process 1**	// **Process 2**

while (1) {
  wait(S);
  printf("Great \n");
  signal(V);
}
	while (1) {
	  wait(V);
	  printf("day \n");
	  printf ("today!! \n");
	  signal(V);
	}

i) Fill table 2, and determine a possible output from the `printf` statements? [3.0]

Table 2: Content of semaphores & output

S	V	Output

ii) Based on the values from table 2, write the sequent of running processes. [2.0]

```
struct semaphore S, V;
S = 2;
V = 0;
```

```
// Process 1
while (1) {
  wait(S);
  printf("Great \n");
  signal(V);
}
```

```
// Process 2
while (1) {
  wait(V);
  printf("day \n");
  printf ("today!! \n");
  signal(V);
}
```

Process	S	V	Output
(initial)	2	0	-
P_1	1	1	Great
P_2	1	1	day today!!
P_1	0	2	Great
P_2	0	2	Day today!!
P_2	0	2	Day today!!

Another possible answer:

Process	S	V	Output
(initial)	2	0	-
P_1	1	1	Great
P_1	0	2	Great
P_2	0	2	Day today!!
P_2	0	2	Day today!!

Example: Producer-Consumer Problem

Initial values: `mutex = 1, empty = n, full = 0`

Producer

```
do {
    ….
    produce an item in nextp
    ….
    wait(empty); //dec empty cnt
    wait(mutex);
    ….
    add nextp to buffer
    ….
    signal(mutex);
    signal(full);//inc full cnt
} while (TRUE);
```

Consumer

```
do {
    wait(full);     //dec full cnt
    wait(mutex);
    …..
    remove an item from buffer to nextc
    ….
    signal(mutex);
    signal(empty); //inc empty cnt
    ….
    consume the item in nextc
    ….
} while (TRUE);
```

Exercise:



Based on the previous example, fill in the following table with the value of those variables used.

Assume that initial value:

`mutex = 1, empty = 5, full = 0`

Producer :

	wait()	signal()
empty		
mutex		
full		

Consumer :

	wait()	signal()
empty		
mutex		
full		

- The producer will start first since `wait(empty=5)` and consumer with `wait(full=0)`
- Once producer update `full=1`, then consumer proceed with all updated value by producer.

Producer :

	wait()	signal()
empty		
mutex		
full		

Consumer :

	wait()	signal()
empty		
mutex		
full		

Initial value: `mutex = 1, empty = 5, full = 0`

```
wait(empty=5)
        empty=4
wait(mutex=1)
        mutex=0
```

```
wait(full=1)
        full=0
wait(mutex=1)
        mutex=0
```

```
signal(mutex=0)
        mutex=1
signal(full=0)
        full=1
```

```
signal(mutex=0)
        mutex=1
signal(empty=4)
        empty=5
```

Producer :

	wait()	signal()
empty	4	
mutex	0	1
full		1

Consumer :

	wait()	signal()
empty		5
mutex	0	1
full	0	



6.6.3 Deadlock and Starvation

- Incorrect use of semaphore operations can produced:
  - **Deadlock** – two or more processes are waiting indefinitely for an event that can be caused by only one of the waiting processes
  - **Starvation** – **indefinite blocking**
    - A process may never be removed from the semaphore queue in which it is suspended



Summary

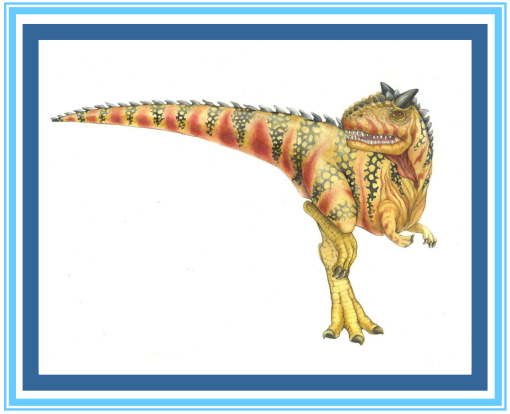
- Mutual exclusion
  - Prevents deadlock
  - Maintained with `test-and-set`, `WAIT` and `SIGNAL`, and semaphores (P, V, and `mutex`)
- Synchronize processes using hardware and software mechanisms

EXERCISE (DIY!!!)

Figure 1 shows the implementation of `Producer-Consumer` using `Semaphores`. Supposed the initial values of the buffer size, `N=8, mutex=1, empty=6` and `full=2`. Complete the value of the appropriate `mutex` variables in Table below if the following sequence of order is executed: *consumer*, *consumer*, *producer*. [*6 marks*]

Process	Wait(full)	Wait(mutex)	Signal (mutex)	Signal (empty)
Initia Value	2	1		6
Consumer				
Consumer				
Process	Wait(empty)	Wait(mutex)	Signal (mutex)	Signal (full)
Producer				

End of Chapter 6



Silberschatz, Galvin and Gagne ©2013